



GENERALITAT
VALENCIANA

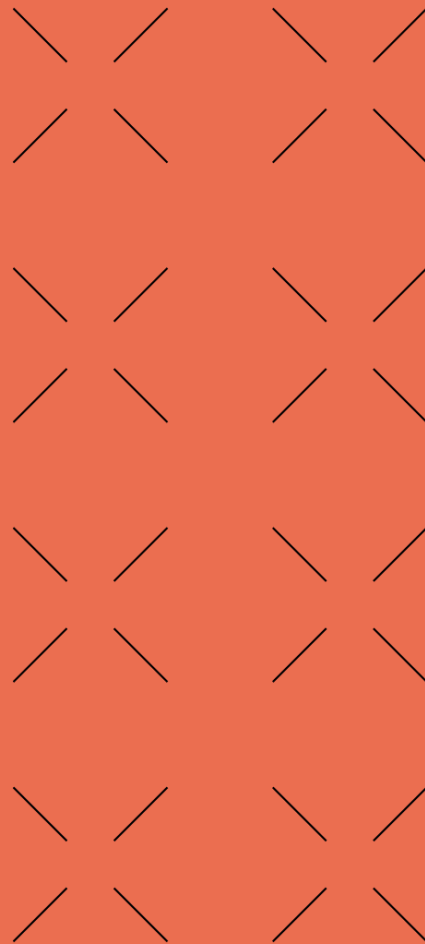
Conselleria d'Educació,
Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 01 — Introducció al pensament computacional Bloc 01

1r CFGS DAW · Programació

Sergi García Barea



L'ordinador és extraordinàriament ràpid

... però absolutament ximple



Un processador modern fa **mil milions d'operacions** per segon... però **no entén** ni tan sols una paraula en valencià.

- El maquinari només entén **binari**: 0 i 1
- Cada 0 i 1 és un **senyal elèctric**: apagat o encès
- El conjunt de 0s i 1s que la màquina entén és el **codi màquina**
- Ningú programa en binari — per això inventem **llenguatges**

La diferència entre un processador ràpid i un de lent és la **velocitat**, no la **intel·ligència**.
L'ordinador no "pensa": executa ordres.

Què és un llenguatge de programació?

- 1 El teu cervell té una **idea** (resoldre un problema)
- 2 El llenguatge és el **pont** entre eixa idea i el processador
- 3 Escris el codi en el llenguatge → un **traductor** ho converteix en codi màquina

*El llenguatge de programació és simplement el **vocabulari**. El repte real és saber **què li has de dir** a la màquina.*

Llenguatges alts vs baixos

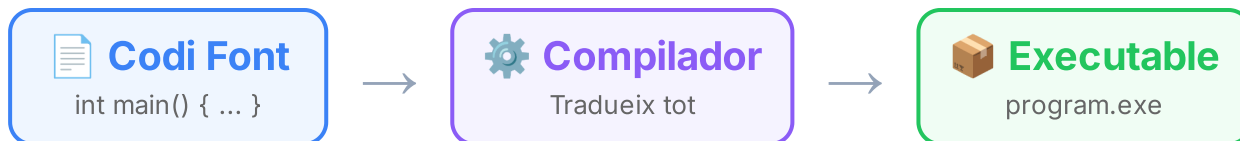
Quina diferència hi ha?

Característica	Llenguatge Alt	Llenguatge Baix
Proximitat al humà	✓ Proper (sembla anglès)	✗ Llunyà (només 0s i 1s)
Velocitat d'execució	✗ Més lent	✓ Més ràpid
Facilitat d'ús	✓ Fàcil d'aprendre	✗ Molt difícil
Exemples	Java, Python, C#, JavaScript	Assembler, C (part)
Portable?	✓ Sí (funciona en qualsevol SO)	✗ No (depèn del processador)

Nosaltres treballarem amb **Java**: un llenguatge alt, portàtil i amb una comunitat enorme.

El Traductor: Compiladors

Tradueixen tot el codi d'una vegada



C, C++, Go: compilen tot el codi → executable independent · **Avantatge:** execució ràpida · **Inconvenient:** cal recompilar per a cada plataforma

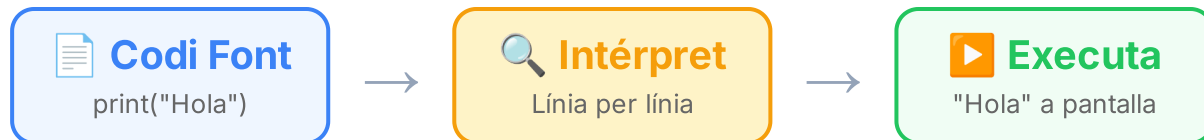
Exemple real: C

```
/* hello.c */
#include <stdio.h>
int main() {
    printf("Hola món!");
    return 0;
}
```

```
$ gcc hello.c -o hello
$ ./hello
Hola món!
```

El Traductor: Intèrprets

Tradueixen línia per línia



Python, JavaScript, Ruby: l'intèrpret executa directament el codi font · **Avantatge:** no cal compilar, provatura ràpida · **Inconvenient:** més lent (traduïx cada vegada)

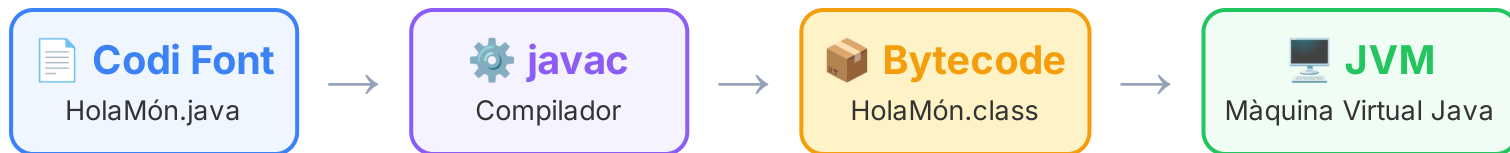
Exemple real: Python

```
# hello.py
print("Hola món!")
nom = input("Com et dius? ")
print("Hola, " + nom + "!")
```

```
$ python hello.py
Hola món!
Com et dius? Maria
Hola, Maria!
```

Java: el millor dels dos mons

El model híbrid



- 1 **Compiles** el codi font → bytecode (format intermedi)
- 2 **LJVM interpreta** el bytecode i l'executa en qualsevol ordinador

"Write once, run anywhere" — Escris una vegada, executes a qualsevol lloc. 🎯

Conceptes Clau: Codi Font i Sintaxi

Codi Font

El **text** que escrius en un editor de codi. És el teu "original".

```
public class HolaMón {  
    public static void main(String[] args) {  
        System.out.println("Hola!");  
    }  
}
```

Sintaxi

Les **regles** del llenguatge. Si la sintaxi és incorrecta, el compilador dona error.

- Cada instrucció acaba amb `;`
- Blocs entre `{ }`
- Distingeix majúscules: `String` ≠ `string`

Conceptes Clau: IDE i Algorisme



Integrated Development Environment — L'editor de codi on treballes.

- Escriure codi amb color
- Detectar errors abans de compilar
- Executar i depurar
- **Exemple:** IntelliJ IDEA, VS Code



Una **seqüència finita de passos** per resoldre un problema.

- Recepta de cuina = algorisme
- Instruccions de muntatge = algorisme
- El cor del programador
- **Important:** Es pot escriure en qualsevol idioma

Resum: Programar = escriure un **algorisme** en un **llenguatge** utilitzant un **IDE**, respectant la **sintaxi**.

Programar NO és memoritzar codi

✗ MENTIDA

"Programar és memoritzar comandaments i codi"

✓ VERITAT

"Programar és **entrenar la capacitat de resoldre problemes**"

- Un bon programador **no té tot el codi memoritzat** — sap on buscar-lo
- El que importa és la **capacitat lògica**, no la memòria
- Google ens dona respostes, però **no sap quin problema estàs resolent**



Analogia: El metge

Un metge **no memoritza tots els llibres de medicina**. Sap on buscar la informació, però el que realment importa és la seua **capacitat de diagnosticar** — connectar símptomes amb causes, pensar en el pacient concret, prendre decisions. Programar és igual: la lògica és la clau, no la memòria.

El Miratge de la Comprensió

L'Il·lusió de la Competència



Mode Espectador

Mires un vídeo o llegeixes un exemple resolt.

"Això és fàcil, ho entenc!"

El cervell està en **mode reconeixement**: passiu, còmode, sense esforç.



Mode Generador

T'asseus davant la pantalla buida i intentes fer-ho.

"On comence?..."

El cervell està en **mode generació**: actiu, difícil, però és on aprens de veritat.



Exemples quotidians



Música

Mires un pianista → "És fàcil!"

T'asseus al piano → "Què és açò?"



Cuina

Mires un xef → "Ho faig!"

Llances oli → "On era el foc?"



Programació

Mires un tutorial → "Ho entenc!"

Pantalla buida → "On comence?"

Entendre l'exemple d'un altre **no és el mateix** que poder fer-ho tu sol. Són dos circuits cerebrals completament diferents.

L'Analogia del Pianista

- 1 **Mires un piano** i penses: "Això no és tan complicat. Prem les tecles en l'ordre correcte."
- 2 **Et poses davant** i intentes... però les mans no obeeixen. Prems les tecles equivocades.
- 3 **Tornes a mirar** el vídeo del pianista. "Jo sí que sé quines tecles són!"
- 4 **La realitat:** Saber què fer i saber-ho fer són habilitats completament diferents.

 **Consell:** No mires més vídeos de com programar. **Asseu-te i intenta fer-ho.** És l'única manera d'aprendre de veritat.

La pantalla buida és el teu hàbitat natural



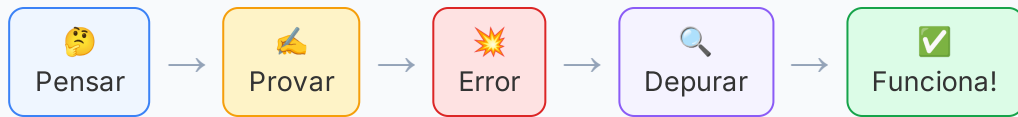
Quan et sents davant d'una pantalla buida i no saps com començar, **no és un senyal d'error**. És el moment normal de tot programador.

- La **pantalla buida** = el punt de partida de tot projecte real
- Fins i tot els programadors professionals **no saben com començar** al principi
- La diferència: ells **proven i fallen** fins que troben el camí

La sensació de no saber què fer és completament normal. Si sempre sabesses què fer, no estaries aprenent res de nou.



El procés real d'un programador



L'Infern del Tutorial

Tutorial Hell

- 1 **Fases de l'infern:** Mires un tutorial → Entens cada pas → Penses "jo puc fer-ho"
- 2 **El parany:** Tanques el tutorial i intentes fer-ho tu... i **no recordes res**
- 3 **El cicle viciós:** Tornes a obrir el tutorial, copies el codi, canvies unes paraules... **progrés fals**
- 4 **La solució:** No copies. **Entén** el concepte → Tanca'l → Intenta fer-ho sol → Si et bloques, busca **una pista**, no la resposta completa



Copiar codi sense entendre'l és com **copiar els deures d'un company**: el professor veu que no ho has fet tu.

Regla de la Pantalla Coberta

✗ MÈTODE INCORRECTE

1. Llegeixes l'exemple
2. Mires el codi mentre tentes fer-ho
3. Cada vegada que et bloques, mires
4. **Resultat:** No aprens res

✓ MÈTODE CORRECTE

1. Llegeixes l'exemple fins que l'entens
2. **Cobreixes la pantalla** (literalment)
3. Intentes reescriure-ho de memòria
4. **Resultat:** realment aprens

✓ **Prova-ho ara mateix:** Llegeix 5 línies de codi → Cobreix la pantalla → Intenta reescriure-les. Funciona sempre.

El Miratge de la Literalitat

L'ordinador no entén matisos

- 1 Si li dius "**posa la sal**", l'ordinador no sap **quanta sal** ni **a on** posar-la
- 2 Si li dius "**prepara una truita**", no sap si vols de patata, de confitura, o si has de comprar els ingredients
- 3 L'ordinador fa **EXACTAMENT** el que li dius, ni més ni menys

Els programadors novells donen per **enteses** molts passos que l'ordinador **no pot endevinar**. Cal ser **explícit** fins a l'extrem.



Exemples de literalitat

✗ El que dius (humà):

- "Posa-ho en un lloc adequat"
- "Fes-me un café"
- "Neteja l'habitació"



✓ El que l'ordinador necessita:

- "Posa el llibre a l'estanteria A, prestatge 3"
- "Prem el botó de café, espera 10s, agafa la tassa"
- "Agafa les peces de roba del terra, posa-les en la cistella"

El Camí Sagrat del Programador

4 passos abans de tocar el teclat

- 1 Entén el problema** — Explica'l amb les teues paraules. Si no el pots explicar, no l'entens.
- 2 Pensa els passos** — Què ha de fer l'ordinador, pas a pas, en llenguatge humà (= Algorisme)
- 3 Escriu en paper** — Resol el problema en el teu llenguatge, sense cap sintaxi de programació
- 4 Tradueix a Java** — Només ara, preocupa't per la sintaxi. El difícil ja està fet!



Per què funciona aquest mètode?

✗ Sense el Camí Sagrat:

- Barreges lògica i sintaxi alhora
- Trobes 3 errors alhora
- No saps quin és de lògica i quin de sintaxi

✓ Amb el Camí Sagrat:

- Lògica primer, sintaxi després
- Quan programes, ja saps QUÈ fer
- El 80% del problema ja està resolt en paper

Exemple: El Camí Sagrat

Problema: Sumar dos números

1 Entén

"El programa demana dos nombres a l'usuari i mostra la seua suma"

2 Pensa

Demana A $\rightarrow$ Demana B $\rightarrow$ Calcula
A+B $\rightarrow$ Mostra resultat

3 Paper

Demana número A

Demana número B

Suma = A + B

Mostra "La suma és: " + Suma

4 Tradueix a Java

```
Scanner teclat = new Scanner(System.in);  
System.out.print("Introdueix un número: ");  
int A = teclat.nextInt();  
System.out.print("Introdueix un altre: ");  
int B = teclat.nextInt();  
int suma = A + B;  
System.out.println("La suma és: " + suma);
```

Exercici: El Robot Literal — Fer café



Dona les instruccions a un robot que **NO entén conceptes abstractes**. Només entén accions físiques concretes.

Intent humà (incorrecte): "Fes-me un café."

Intent robot (correcte):

1. Agafa la tassa de l'armari
2. Posa-la sota l'obrador de café
3. Prem el botó "café sol"
4. Espera 10 segons
5. Agafa la tassa
6. Porta-la a la taula

Exercici: El Robot Literal — Motxilla



Instruccions perquè el robot prepare la motxilla de classe. Que no es quede res!

Intent humà: "Prepara la motxilla."

Intent robot (exemple):

1. Obri l'armari
2. Agafa el llibre de Matemàtiques
3. Posa'l dins la motxilla
4. Agafa el llibre de Programació
5. Posa'l dins la motxilla
6. Agafa el llapis i el bolígraf
7. Posa'ls dins l'estoig
8. Tanca l'armari
9. Comprova que la motxilla té: 2 llibres, 2 bolígrafs, 1 llapis

Exercici: El Robot Literal — Truita



Això és més complicat: cal **comprar, pelar, tallar, fregir, batre, girar**. I cal decidir QUANTITATS.

Exemple (parcial):

1. Agafa 4 patates de la nevera
2. Pela-les
3. Talla-les en rodanxes fines (2mm)
4. Agafa una paella gran
5. Posa-hi oli fins a cobrir el fons
6. Encén el foc a mitjana potència
7. Espera 2 minuts (fins que l'oli estiga calent)
8. Afig les patates
9. Fregir 15 minuts (remenant cada 3 minuts)
- ... (continua)

Els 5 Dimonis de la Lògica

Errors que tot programador comet



1. Falten passos

Donar per fet alguna cosa



2. Ordre incorrecte

Passos fora de seqüència



3. Ambigüitat

Instruccions vagues



4. Info implícita

Assumir coneixement previ



5. Decisions no considerades

No pensar en "i si..."



Exemple real: Fer una truita

✗ Amb els 5 dimonis:

"Pela les patates, talla-les, posa oli, fregix-les..." — Quantes?
Quant d'oli? Temperatura?



Sense dimonis:

"Pela 4 patates, talla-les en rodanxes de 2mm, posa 200ml d'oli, foc mitjà..." — Tot explícit i concret.

Dimoni 1 i 2

✗ Falta un pas

"Obri el microones, posa el plat, tanca la porta"

On està **prémer el botó**? On està **el temps**?

✗ Ordre incorrecte

"Tanca la porta, obri el microones, posa el plat"

No pots tancar la porta **abans** d'obrir-la!



Consell: Llig les instruccions en veu alta. Si et sonen estranyes, probablement hi ha un error de lògica.

Dimonis 3 i 4

✗ Ambigüitat

"Posa-ho en un lloc adequat"

Quin lloc? La taula? L'armari? El terra? L'ordinador no pot decidir.

✗ Info implícita

"Afegeix la sal"

Quina sal? Quantitat? On la poses? L'ordinador no sap que tens sal a la cuina.

Els programadors **assumeixen** molta informació que l'ordinador **no té**. Cal ser explícit amb tot.

Dimoni 5



Quan escrivim un algorisme, sovint **no pensem en el que pot fallar**.

✗ Sense decidir

"Divideix A entre B"

I si B és zero? No ho has considerat!

✓ Amb decisió

"Si $B = 0$, mostra error. Sinó, divideix A entre B."

Has previst el cas problemàtic!

✓ **Regla d'or:** Per cada acció, pregunta't: "**I si...?**" (i si és zero? i si és negatiu? i si no existeix? i si és buit?)

Com es menja un elefant?

Boca per boca! 

- 1 Un **problema enorme** és simplement una col·lecció de **problemes xicotets**
- 2 Si resols cada problema xicotet **un per un**, al final has resolt el problema sencer
- 3 La **descomposició** és l'habilitat més important d'un programador

 **Exemple real:** "Crear una app de biblioteca" sembla impossible. Però "fer un botó de cercar" és fàcil. I "emmagatzemar un llibre en una llista" també. **Són només peces!**

Exemple: Biblioteca Municipal

Descomposició en subsistemes



Gestió d'Usuaris

Alta, baixa, cerca, dades personals



Catàleg de Llibres

Afegir, eliminar, cercar per títol/autor



Sistema de Préstec

Prestar, tornar, multes, dates



Cerca i Filtres

Per gènere, autor, estat, disponibilitat



Informes

Préstecs del mes, llibres més prestats



Pagament de Multes

Calcular, cobrar, historial



Clau: Cap dels subsistemes és "impossible". Cadascun es pot programar **independentment** i després unir-los.

Subsistema de Préstec

Descomposició detallada (6 passos)

- 1 Identificar l'usuari** — Introduir DNI o número de targeta
- 2 Identificar el llibre** — Escanejar codi de barres o cercar per títol
- 3 Comprovar multes pendents** — Si en té, no pot prestar
- 4 Comprovar disponibilitat** — El llibre ha d'estar "lliure", no "prestat"
- 5 Registrar dates** — Data de préstec i data de tornada (15 dies)
- 6 Canviar estat del llibre** — De "lliure" a "prestat" al catàleg

Flux visual del procés



Exemple pràctic

Descomposar "Fer una truita"



Bloc: Compra

- Llistar ingredients
- Anar al supermercat
- Comprar patates, ous, oli, sal
- Pagar



Bloc: Preparació

- Pelar les patates
- Tallar-les en rodanxes
- Batre els ous amb sal



Bloc: Cocció

- Fregir les patates (oli, foc mitjà)
- Escórrer l'excés d'oli
- Mesclar patates amb ou batut
- Cuinar la truita per les dues cares



Bloc: Servir

- Emportar-la al plat
- Deixar refredar 2 minuts
- Servir

Les 3 Regles d'Or de la Descomposició

#	Regla	Exemple
1	Mai no resolgues dos sub-problemes alhora	No intentes "comprar i cuinar" alhora. Primer compra, després cuina.
2	Si un sub-pas encara és difícil, torna a dividir-lo	"Cuinar la truita" és massa ampli → divideix en fregir, mesclar, girar.
3	Celebra les xicotetes victòries	Cada pas que funciona et dona seguretat per al següent.

*Si una tasca et fa por, és simplement perquè **no l'has dividit en peces prou xicotetes**.* ✂️🧩

Regles 1 i 2 en acció

Regla 1: Un pas a la vegada

✗ Incorrecte

"Llig el llibre, compra els ingredients, pela les patates i cuina la truita tot alhora"

✓ Correcte

Primer: llig la recepta. Després: compra. Després: pela.
Després: cuina.

Regla 2: Divideix més si cal

✗ Massa ampli

"Cuinar la truita" — Encara és massa complicat!

✓ Dividit

1. Fregir patates → 2. Escórrer → 3. Mesclar amb ou → 4.
Cuinar per les dues cares

Regla 3: Celebra les victòries



Cada sub-problema que resols és un **pas endavant**. Celebra'li!

- Has fet que el botó de "cercar" funcione? **Victòria!** 🎯
- Has emmagatzemat un llibre en una llista? **Victòria!** 🎯
- Has comprovat que un DNI és vàlid? **Victòria!** 🎯



Per què? Perquè cada victòria et dona **confiança** i **momentum** per al següent pas. Programar és una maratón, no un sprint.

Resum — Bloc 01 Completat!

El que hem après

Secció 0

Fonaments tècnics

Binari, llenguatges, compiladors, JVM, IDE, algorisme

Secció 1

Mentalitat

No memoritzar, miratge comprensió, pantalla coberta

Secció 2

El Camí Sagrat

Literalitat, 4 passos, robot literal, 5 dimonis

Secció 3

Descomposició

Dividir problemes, biblioteca, 3 regles d'or



Ja tens la mentalitat del programador! Ara toca posar-la en pràctica amb codi real. 

